

Prereq: Lab 05: The Infrastructure Pipeline

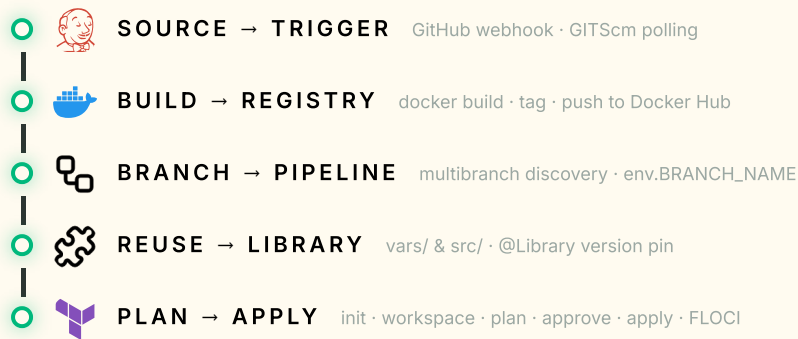
 Hands-on lab · 180 min · Env: See book overview · Overview & prereqs · Download PDF · [Reset checks](#)

CI/CD SYSTEMS · PIPELINE CONTROL ROOM · JENKINS LABS

JENKINS LABS

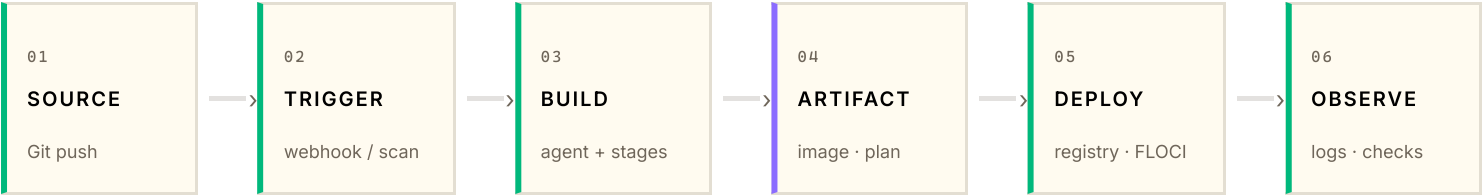
Five hands-on labs — webhooks to multibranch to shared libraries to Docker & Terraform on FLOCI. Every push verified, every artifact tagged, every stage observable.

WEBHOOK TRIGGER · DOCKER BUILD & PUSH · MULTIBRANCH · SHARED LIBRARY · TERRAFORM + FLOCI



05 LABS · 05 FLOWS · ONE CONTROL ROOM

Nabawy



01 / CONTENTS

- 1. Lab 01 — GitHub Webhook Trigger p. 03 — push → webhook → Jenkins build
- 2. Lab 02 — Docker Build, Tag & Push p. 06 — Jenkinsfile → Docker Hub → FLOCI deploy
- 3. Lab 03 — Multibranch Pipeline p. 09 — main / dev / stg · one Jenkinsfile · when
- 4. Lab 04 — Shared Library p. 12 — Global Library · UatInput · scripted + declarative
- 5. Lab 05 — Terraform on FLOCI p. 15 — init → workspace → plan → approve → apply
- 6. Appendix — Proofs & Troubleshooting p. 18 — what green proves · failure map

02 / LAB FORMAT — OBSERVE · EXECUTE · VERIFY · EXPLAIN

Every lab follows the control-room loop: **OBSERVE** what is currently happening, **EXECUTE** a precise action, **VERIFY** via logs/console/registry/scan log, **EXPLAIN** why. Each lab ends with **PROOF** — what the experiment demonstrates about the system.

PRE-FLIGHT RULE

Step 0 of every lab gathers tools, credentials, repo state, and user inputs **before** any mutation. No mid-flow surprises.

PRINTING THIS BOOK

Open in browser → Ctrl/Cmd+P → Save as PDF → Background graphics ON → A4. Colors carry state and survive grayscale because every state pairs **color + symbol + label**.

CONTINUITY

Lab 01 wires the trigger. Lab 02 builds the artifact that trigger produces. Lab 03 multiplies the pipeline across branches. Lab 04 extracts reuse. Lab 05 applies the same trigger-build-approve pattern to infrastructure. The same lifecycle recurs — only the payload changes.

01

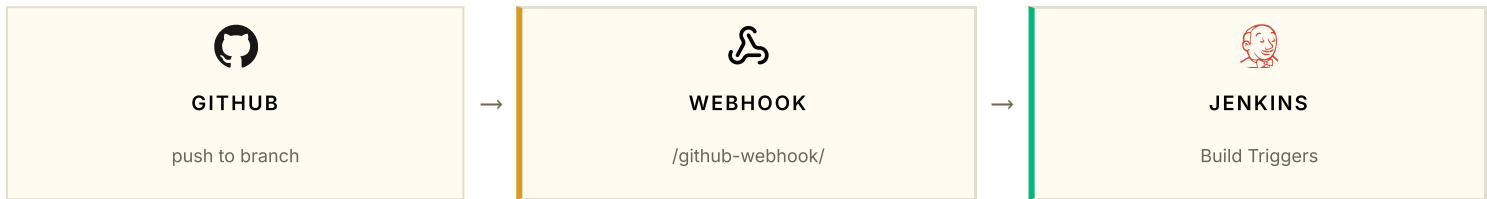
GITHUB WEBHOOK TRIGGER

A push should be enough. **One commit, one webhook, one build — no human clicking Build Now.** This lab removes the manual step between GitHub and Jenkins.

01 / WHAT THIS LAB PROVES

PROOF — Jenkins reacts to Git events via `/github-webhook/` and **GitHub hook trigger for GITScm polling**. Reachability + credentials + branch match are the only three things that can break it.

02 / ARCHITECTURE



GitHub delivers a POST to Jenkins. Jenkins matches the repo + branch, checks out with stored PAT, and starts the job. If any hop is unreachable or mismatched, GitHub's **Recent Deliveries** shows the failure code — that is the first place to look.

03 / STEP 0 — PRE-FLIGHT (BEFORE STEP 1)

Check	Command / Location	Decision
Tools	<code>curl -s -o /dev/null -w "%{http_code}" http://localhost:8080</code> <code>· gh --version / gh auth status</code>	If Jenkins only on localhost/WSL → tunnel decision belongs here
Credentials	GitHub PAT (repo scope) · Manage Jenkins → Credentials existing token	Reuse if present; PAT creation is browser-only — never mid-flow
Repo state	Existing job? existing webhook (Settings → Webhooks)? default branch name?	Do not assume <code>main</code> ; read it from the repo

ASK IN ONE BATCH — BEFORE STEP 1

Only ask for what is genuinely missing after the checks: repo URL + branch, valid PAT (or generate one), Jenkins public URL / tunnel choice, job name. Then proceed uninterrupted.

04 / STEPS 1-4 — JOB + CREDENTIALS + CHECKOUT



01 — Pipeline job

Reuse if exists; else **New Item** → **Pipeline** (`github-webhook-pipeline`). Paste a placeholder script, save, run once to prove the job itself is green before wiring Git.



02 — Repository URL

Copy HTTPS clone URL: `https://github.com/<user>/<repo>.git`.



03-04 — Credentials via Pipeline Syntax generator

Pipeline Syntax → **checkout: Check out from version control** → **SCM: Git** → paste URL → **Add** → **Jenkins** → **Username with password** (username = GitHub user, password = PAT) → ID `github-token` → select it → set **Branch to build** to the confirmed branch → **Generate Pipeline Script** → paste snippet into job → save → run to confirm checkout pulls the repo.

```
# generated snippet (example - use the one Jenkins generates for you)
checkout([$class: 'GitSCM',
  branches: [[name: '*/main']],
  userRemoteConfigs: [[url: 'https://github.com//.git',
    credentialsId: 'github-token']]])
```

05 / STEPS 5-6 — WIRE BOTH SIDES



05 — GitHub Webhook

Repo → Settings → Webhooks → Add webhook → Payload URL `http://<jenkins-url>/github-webhook/` (trailing slash matters) → `application/json` → push events → Save → confirm green checkmark (initial ping delivered). If it exists and is green, reuse it.



06 — Jenkins trigger

Job → Configure → Build Triggers → GitHub hook trigger for GITScm polling → enable → Save.

WHY THIS EXISTS

Without a webhook, Jenkins only learns about pushes when someone clicks Build Now or a poll discovers them late. The webhook moves discovery from **minutes/hours** to **seconds**.

06 / STEP 7 — END-TO-END TEST

```
$ echo "<!-- trigger -->" >> index.html && git commit -am "trigger webhook" && git push
# then watch Jenkins
→ Jenkins build history: new build starts automatically
→ Console Output: checkout pulled latest commit SHA
```



VERIFY — Jenkins build history shows an automatic build.

Console output confirms the checkout SHA matches the pushed commit. GitHub webhook **Recent Deliveries** shows 200.

● PASSED

GOTCHA

WSL / localhost Jenkins is invisible to GitHub. Webhooks require a public URL — use `ngrok / localhost.run` or a reverse proxy. Decide in Step 0. **Branch mismatch** (`main` vs `master`) also silently breaks checkout — confirm the real default branch.

PROOF — A file edit + push triggered a build without **Build Now**. The loop **commit → webhook → build** is closed. • WEBHOOK
PROVED

02

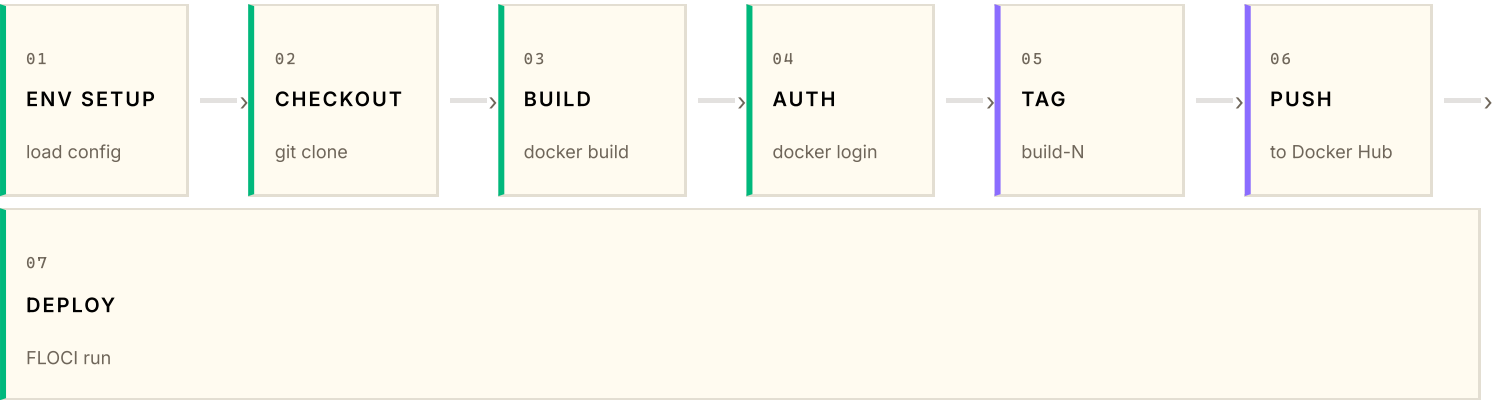
DOCKER BUILD & PUSH

The artifact is the contract. Build it once, tag it immutably, push it to a registry, run the same image everywhere — including FLOCI.

01 / WHAT THIS LAB PROVES

PROOF — Jenkins can produce a Docker image, authenticate via stored credentials, tag with `build-${BUILD_NUMBER}`, push to Docker Hub, and optionally run it inside a **FLOCI** simulated AWS environment — without hardcoding secrets.

02 / FLOW — SEVEN STAGES



03 / STEP 0 — PRE-FLIGHT

Check	Command / Location	Decision
Tools	<code>docker --version</code> · Jenkins Docker Pipeline + Credentials Binding plugins	Confirm Jenkins user can run docker (group / socket / DiD)
Credentials	Manage Jenkins → Credentials → Docker Hub credential ID	Reuse; else need username + access token (not password)
Repo state	<code>hello.py</code> · <code>requirements.txt</code> · <code>Dockerfile</code> · <code>Jenkinsfile</code>	Create/commit missing pieces before pipeline

ASK IN ONE BATCH

Docker Hub namespace (myusername/app), tagging convention (build-\${BUILD_NUMBER}), whether FLOCI deploy is in scope (+ endpoint), repo URL/branch.

04 / STEPS 1-2 — REPO + CREDENTIALS

☐

PREPARE REPO — `hello.py` · `requirements.txt` · `Dockerfile` · `Jenkinsfile`.

Commit and push.

☐

CREDENTIALS — **Manage Jenkins** → **Credentials** → Username with password.

Username = Docker Hub user, Password = access token, ID dockerhub-creds.

05 / STEP 3 — JENKINSFILE (SEVEN STAGES)

```
pipeline {
  agent any
  environment {
    IMAGE_NAME = "myusername/app"
    IMAGE_TAG  = "build-${env.BUILD_NUMBER}"
  }
  stages {
    stage('Environment Setup') { steps { echo 'Loading environment...' } }
    stage('Code Checkout')     { steps { checkout scm } }
    stage('Docker Build')      { steps { sh "docker build -t ${IMAGE_NAME}:local ." } }
    stage('Authentication') {
      steps {
        withCredentials([usernamePassword(credentialsId: 'dockerhub-creds',
          usernameVariable: 'DOCKER_USER', passwordVariable: 'DOCKER_PASS')]) {
          sh 'echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin'
        }
      }
    }
    stage('Image Tagging') { steps { sh "docker tag ${IMAGE_NAME}:local ${IMAGE_NAME}:${IMAGE_TAG}" } }
    stage('Image Push')    { steps { sh "docker push ${IMAGE_NAME}:${IMAGE_TAG}" } }
    stage('Application Deployment') {
      when { expression { return params.DEPLOY_TO_FLOCI == true } }
      steps { echo "Deploying ${IMAGE_NAME}:${IMAGE_TAG} into FLOCI..." }
    }
  }
}
```

GOTCHA

Never hardcode the token in the Jenkinsfile. Use `withCredentials`. Prefer a scoped **access token** over the account password.

06 / STEPS 4-5 — RUN & VALIDATE



JOB — Pipeline → Pipeline script from SCM (reads Jenkinsfile from repo) + webhook trigger (Lab 01).
Push a commit → pipeline runs automatically.

```
# expected stage progression
[01] Environment Setup      • PASSED
[02] Code Checkout         • PASSED
[03] Docker Build          • PASSED
[04] Authentication       • PASSED
[05] Image Tagging         • PASSED
[06] Image Push            • PASSED
[07] Application Deploy    ◦ SKIPPED when DEPLOY_TO_FLOCI == false
```



- VERIFY —**
- Docker Hub shows `myusername/app:build-<N>` under the namespace.
 - Second push → `build-<N+1>` — tag increments with `BUILD_NUMBER`.
 - If FLOCI in scope → simulation reflects the running `hello.py`.

PROOF — The same commit SHA built once, tagged immutably, and pushed as a reusable artifact. Registry is the handoff to deploy. • ARTIFACT PROVED

07 / STATE TRANSITIONS

State	Meaning	Recovery
● BUILDING	<code>docker build</code> in progress	Watch agent executor, Docker daemon logs
● PUSHED	Image visible on Docker Hub	Pull locally: <code>docker pull myusername/app:build-N</code>
● FAILED	<code>docker login</code> or push denied	Check credential ID, token scope, repo visibility

03

MULTIBRANCH PIPELINE

One job, three branches, one Jenkinsfile. **Branches are discovered, not configured — and when decides what runs where.**

01 / WHAT THIS LAB PROVES

PROOF — A **Multibranch Pipeline** is a *folder of branch jobs* driven by branch discovery + **Jenkinsfile** on each branch. `env.BRANCH_NAME` makes one file behave as three pipelines. Deleting a branch orphans its job on next scan.

02 / CONCEPT — FOLDER OF JOBS

MAIN	Jenkinsfile found → build	main
DEV	Jenkinsfile found → build	dev
STG	Jenkinsfile found + when(stg) runs	stg

Parent job controls everything: Branch Sources, **Filter by name** (main dev stg), and **Additional Behaviors**. Individual branch items are not directly configurable — they inherit.

03 / STEP 0 — PRE-FLIGHT

Check	Command / Location	Decision
Tools	git --version · Git + Pipeline: Multibranch plugins	Air-gapped → periodic/manual scans, not webhooks
Credentials	Public repo → none; private → Jenkins Git credential (SSH / token)	Verify git ls-remote <url> before wiring
Repo state	Branches main / dev / stg exist? Jenkinsfile on any branch? existing job name?	Name the multibranch job after the repo

DESIGN CHOICE

This lab uses the **generic Git branch source** (not GitHub-specific), even if the repo is hosted on GitHub. It proves Jenkins works with any Git remote and forces explicit include filters.

04 / STEPS 1-4 — CREATE & SCAN

☐

01 — New Item → Multibranch Pipeline

Name = repo name. Click OK.

☐

02 — Branch Source → Git

URL = clone URL · Credentials = None (public) or the private-repo credential.

☐

03 — Filter + Behaviors

Behaviors → Filter by name (with wildcards) Include: main dev stg · Exclude: blank. Add: Check out to matching local branch + Clean before/after checkout.

☐

04 — Save → observe scan log

Scan Multibranch Pipeline Log reports Jenkinsfile not found if the file is not yet on any branch — expected at this point.

05 / STEPS 5–7 — JENKINSFILE + BRANCHES

```
# Jenkinsfile on default branch (main)
pipeline {
  agent { label 'linux' }
  options { buildDiscarder(logRotator(numToKeepStr: '5')); disableConcurrentBuilds() }
  stages { stage('Hello') { steps { echo 'hello' } } }
}
```



05 — Commit Jenkinsfile to main → push.



06 — Scan Multibranch Pipeline Now

Log now shows Jenkinsfile found on main → build triggered. Optionally set a periodic scan interval.



07 — Create dev + stg from main

git checkout -b dev && git push -u origin dev · same for stg → Scan → all three appear with Jenkinsfile found.

06 / STEP 8 — BRANCH-SPECIFIC BEHAVIOR (STG ONLY)

```
# on branch stg - add a when-guarded stage
stage('Staging Verification (stg only)') {
  when { expression { return env.BRANCH_NAME == 'stg' } }
  steps { sh 'cat README.md' }
}
```



EXPECT —

stg build runs the stage and cats the file; main and dev skip it (● SKIPPED). Visible in stage view / logs.

07 / STEP 9 — ORPHAN CLEANUP

```
# delete a branch remotely, then rescan
git push origin --delete dev # or delete via GitHub UI
# Jenkins → Scan Multibranch Pipeline Now
→ scan log: "orphaned item" → dev job removed from folder
```

GOTCHA

Deletion does not remove the Jenkins job immediately. Only the **next scan** evaluates orphans. No webhook here is intentional — this lab simulates an air-gapped Jenkins.

PROOF — One Jenkinsfile, three jobs, conditional stages by branch, orphan removal on rescan.

• MULTIBRANCH PROVED

04

SHARED LIBRARY

Copy-paste is not reuse. **A pipeline library is a dependency: versioned, reviewed, rolled back.**
One Groovy class, two pipelines, one approval gate.

01 / WHAT THIS LAB PROVES

PROOF — A **Global Pipeline Library** (`src/com/<you>/UatInput.groovy`) exposes `buildIsUatApproved()` (checks `tools/approved.txt`) to both scripted and declarative pipelines via `@Library('shared-library')`. Changing the library changes all consumers — and reverting fixes them.

02 / REPO LAYOUT (JENKINS EXPECTS THIS EXACTLY)

```
(root)
├─ src/
│   └─ com/
│       └─ <yourname>/
│           └─ UatInput.groovy → class UatInput { boolean buildIsUatApproved() }
checks workspace file: tools/approved.txt → true / false
```

03 / STEP 0 — PRE-FLIGHT

Check	Command / Location	Decision
Tools	<code>curl http://localhost:8080 · git --version · gh auth status</code>	Public vs private repo decides credential need
Credentials	Private repo? → Jenkins credential (username + PAT)	Public → no credential in library config
Repo / Jenkins state	Existing <code>jenkins-shared-library</code> repo? existing Global Pipeline Libraries entry? existing pipeline job?	Reuse; confirm default branch name (<code>main</code> / <code>master</code>)

04 / STEPS 1-2 — REPO + REGISTRATION

☐

01 — Create (or reuse) the library repo

Example: `jenkins-shared-library` → add `src/com/<you>/UatInput.groovy` implementing `buildIsUatApproved()` → commit to `master/main` → copy clone URL.

☐

02 — Manage Jenkins → Configure System → Global Pipeline Libraries → Add

Name: `shared-library` · Default version: `master` · Retrieval: **Modern SCM** · SCM: **GitHub** · Project Repository: pasted URL → **Validate** → Apply/Save. Reuse if already pointing at this repo.

```
# UatInput.groovy - minimal logic
package com.example
class UatInput implements Serializable {
    boolean buildIsUatApproved() {
        return fileExists('tools/approved.txt')
    }
}
```

WHY VERSION PIN MATTERS

Default `version = master` is a live dependency. Pin consumers with `@Library('shared-library@main')` or a tag when you want reproducible builds rather than latest.

05 / STEPS 3-4 — SCRIPTED PIPELINE



PIPELINE — New Item → Pipeline (**shared-library-test-pipeline**) → **unchecked Groovy Sandbox**.

Sandbox off avoids approval noise for this simple class (production: use In-process Script Approval instead).

```
@Library('shared-library') _
import com.example.UatInput
def uatInput = new UatInput()
println "Build is UAT approved: ${uatInput.buildIsUatApproved()}"
```



VERIFY — With **tools/approved.txt** present → **true**; without → **false** in Console Output.

06 / STEP 5 — DECLARATIVE PIPELINE (WHEN GATE)

```
@Library('shared-library') _
import com.example.UatInput
def uatInput = new UatInput()
pipeline {
  agent any
  stages {
    stage('Run only if approval exists') {
      when { expression { return uatInput.buildIsUatApproved() } }
      steps { echo 'The build has been approved.' }
    }
  }
}
```



EXPECT — Stage runs when **approved.txt** exists; skipped without failing the build when absent.

07 / STEP 6 — END-TO-END VALIDATION

- With file → ● **PASSED** stage executed.
- Without file → ● **SKIPPED** — no output from inside **when** (not a failure).
- Break the library → both consumers break; revert → both fix (proves shared dependency).
- Trouble: **not found** → name/version mismatch; sandbox blocks → approve signature or restrict source.

PROOF — Reuse without copy-paste, gated by workspace state, consumed from two pipeline styles.

• REUSE PROVED

05

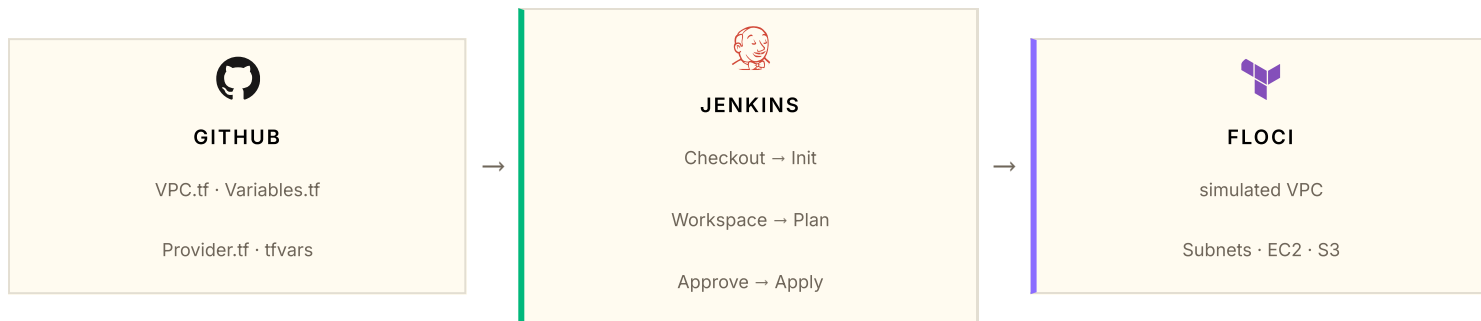
TERRAFORM ON FLOCI

Infrastructure is a pipeline too. **Webhook → init → workspace → plan → human approve → apply → mail on failure — against a simulated AWS.**

01 / WHAT THIS LAB PROVES

PROOF — The same CI/CD loop that builds apps can provision infrastructure: `terraform init / workspace / plan / apply` behind a Jenkins input gate, with `Provider.tf` pointing at **FLOCI** instead of real AWS — safe, cost-free, auditable.

02 / ARCHITECTURE



03 / STEP 0 — PRE-FLIGHT

Check	Command / Location	Decision
Tools	terraform -version · Jenkins Terraform + Pipeline + Mailer plugins	Terraform must exist on the agent that runs sh 'terraform ... '
Credentials	FLOCI endpoint / API key / provider block format — user must supply	Real AWS docs do not apply; FLOCI is the provider target
Repo / state	VPC.tf · Variables.tf · Provider.tf · dev / stg.tfvars / Prod · Jenkinsfile exist?	Workspaces dev / stg / prod exist? backend = local or remote?

ASK IN ONE BATCH

Provider.tf FLOCI block (endpoint + credentials), environments/workspaces + tfvars contents, who approves the input gate, mail recipient for **Fail** → **Mail**, repo URL/branch.

04 / STEPS 1-2 — REPO + JENKINSFILE

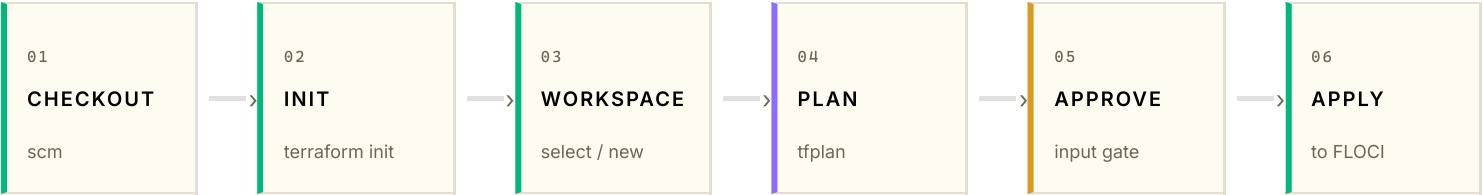


REPO — Ensure at repo root:

VPC.tf · Variables.tf · Provider.tf (FLOCI endpoint) · dev / stg.tfvars / Prod → commit & push.

```
pipeline {
  agent any
  stages {
    stage('Checkout')      { steps { checkout scm } }
    stage('Terraform Init') { steps { sh 'terraform init' } }
    stage('Workspace Select'){ steps { sh 'terraform workspace select ${ENV_NAME} || terraform workspace new ${ENV_NAME}' } }
    stage('Plan')          { steps { sh 'terraform plan -var-file=${TFVARS_FILE} -out=tfplan' } }
    stage('Approve')       { steps { input message: 'Apply this Terraform plan?', ok: 'Approve' } }
    stage('Apply')         { steps { sh 'terraform apply -auto-approve tfplan' } }
  }
  post {
    success { echo 'Provisioned.' }
    failure { mail to: 'team@example.com', subject: "Terraform failed: ${env.JOB_NAME}" }
  }
}
```

05 / EXECUTION FLOW



```
# stage view during approval
[01] Checkout           • PASSED
[02] Terraform Init     • PASSED
[03] Workspace Select   • PASSED  workspace: stg
[04] Plan               • PASSED  Plan: 3 to add, 0 to change
[05] Approve            • WAITING  input: Apply this Terraform plan? [Approve / Abort]
[06] Apply              ○ WAITING

# after Approve →
[05] Approve            • APPROVED
[06] Apply              • PASSED  Simulated VPC / Subnets / EC2 / S3 via FLOCI
```

06 / VERIFY

- Select `stg` workspace → plan shows expected diff for `stg.tfvars`.
- Approval gate pauses; **Abort** skips Apply (plan discarded).
- After Approve → FLOCI reflects simulated VPC + Subnets + EC2 + S3.
- On failure → `post { failure { mail ... } }` fires (if SMTP configured).
- Push a new commit → webhook re-triggers the same pipeline (Lab 01's trigger reused here).

WHY APPROVAL EXISTS

Automation stops where human risk judgment is required. `input` is the amber gate between **knowing what will change** (plan) and **changing it** (apply).

PROOF — Infrastructure under the same lifecycle as application code: versioned, planned, approved, applied, observable — without touching real AWS. • IAC PROVED

01 / WHAT EACH LAB PROVED

Lab	Proved	Key Signal
01 Webhook	Push triggers build automatically	Jenkins build without Build Now + webhook 200
02 Docker	Immutable artifact via registry	myusername/app:build-N on Docker Hub; N increments
03 Multibranch	Branch discovery as jobs	3 jobs, conditional stg stage, orphan removed on rescan
04 Shared Lib	Reuse as versioned dependency	One class → two pipelines; revert fixes both
05 Terraform	Infra as a gated pipeline	Plan → Approve → Apply on FLOCI; mail on failure

02 / FAILURE → RECOVERY MAP

Failure	Where to Look	Fix
Webhook 4xx/5xx	GitHub Recent Deliveries	Public URL / tunnel; trailing slash; webhook secret
Checkout failed	Jenkins Console Output	PAT scope repo ; credential ID; branch name
Docker login denied	Stage Authentication log	Use access token; verify ID dockerhub-creds ; repo visibility
Multibranch empty	Scan log: Jenkinsfile not found	Commit Jenkinsfile to the branch before scanning
Library not found	Pipeline log: Library resolution	Name shared-library must match @Library(...) ; correct default version
Terraform plan diff wrong	terraform plan output	Wrong workspace / tfvars file; Provider.tf still points to real AWS not FLOCI
Apply paused forever	Stage Approve (input)	Someone must click Approve/Abort; check authorized approvers

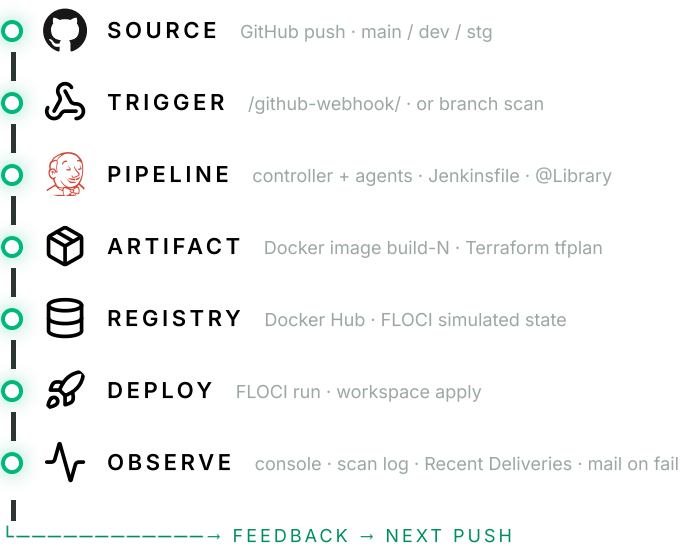
03 / KEY COMMANDS

```
curl -s -o /dev/null -w "%{http_code}" http://localhost:8080
docker --version && docker info
terraform -version && terraform workspace list
gh --version && gh auth status
git ls-remote https://github.com/<user>/<repo>.git
# branch helpers (Lab 03)
git checkout -b stg && git push -u origin stg
git push origin --delete dev && echo "rescan to see orphan removal"
```

CONTINUITY — WHAT COMES NEXT

These five labs compose: **Lab 01's trigger** launches **Lab 02's build**, repeated per **Lab 03's branches**, deduplicated by **Lab 04's library**, and applied to infra by **Lab 05**. The next step is securing the chain — credentials, agents, and permissions — covered in **PDF 16 · Jenkins Security** and **PDF 13 · Credentials**.

04 / JENKINS LABS LOOP



05 / INTERVIEW CHECKS

WHY DOES /github-webhook/ NEED A TRAILING SLASH?

Jenkins registers the endpoint with the slash. Without it GitHub receives a redirect that is not treated as a successful delivery.

MULTIBRANCH VS WEBHOOK?

Multibranch discovers branches by scanning; webhooks notify eagerly. Labs 01 and 03 show both — webhook for immediate trigger, scan for air-gapped/periodic discovery.

WHY withCredentials?

It injects the secret as an env var at runtime, masks it in logs, and avoids storing it in the Jenkinsfile or Git history.

SHARED LIBRARY VERSION PIN?

@Library('lib@tag') freezes the consumer. Unpinned master tracks latest — useful for shared standards, risky for reproducible builds.

RELATED BOOKS

 **Lab 05: The Infrastructure Pipeline**
60 min · **ADV**
→ Previous in path

 **Lab 01: Automated Integration Foundations**
45 min · **INT**
→ Shared tags

 **Lab 02: Container Delivery**
50 min · **INT**
→ Shared tags

 **Lab 03: Multibranch Foundations**
45 min · **INT**
→ Same path